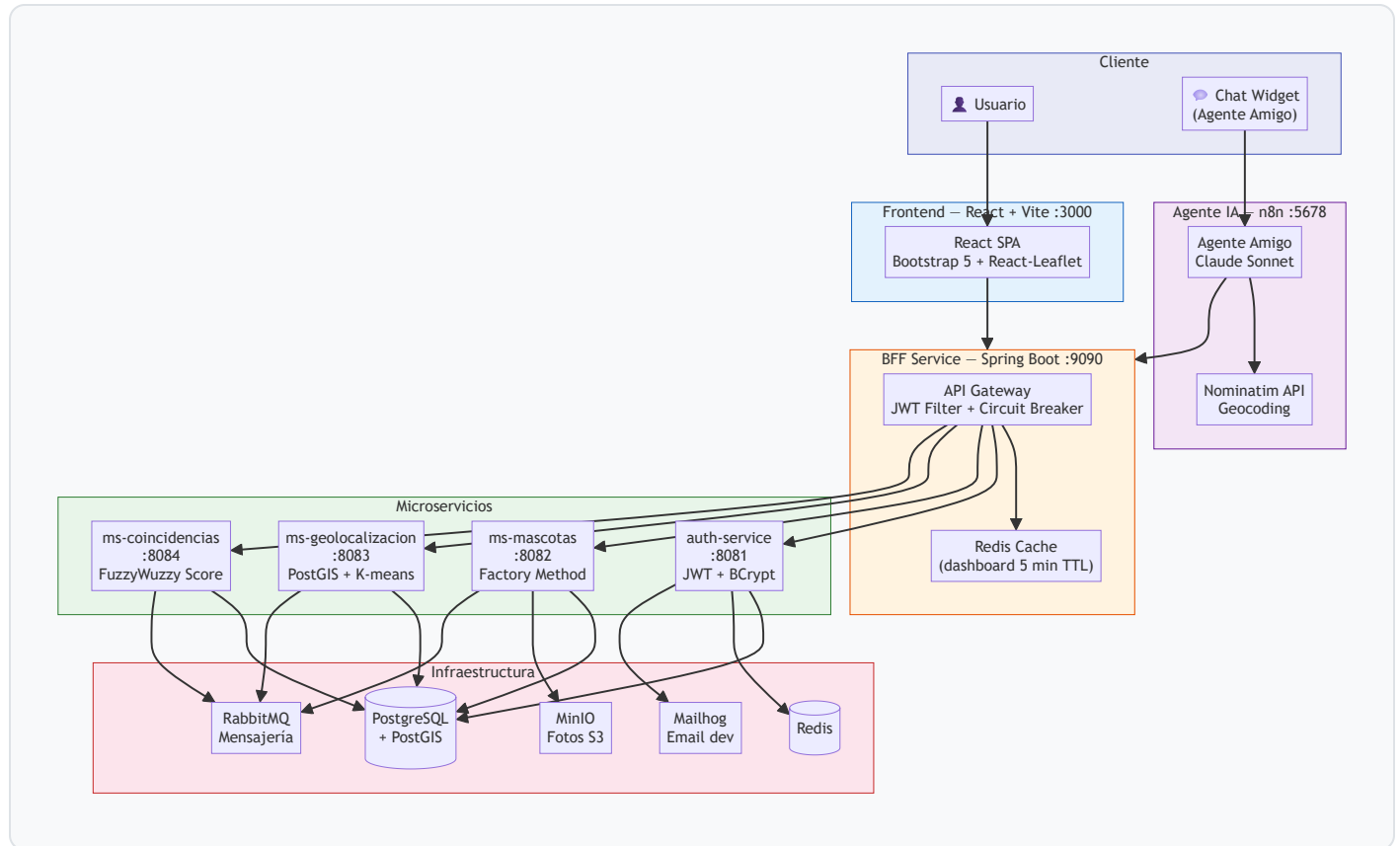


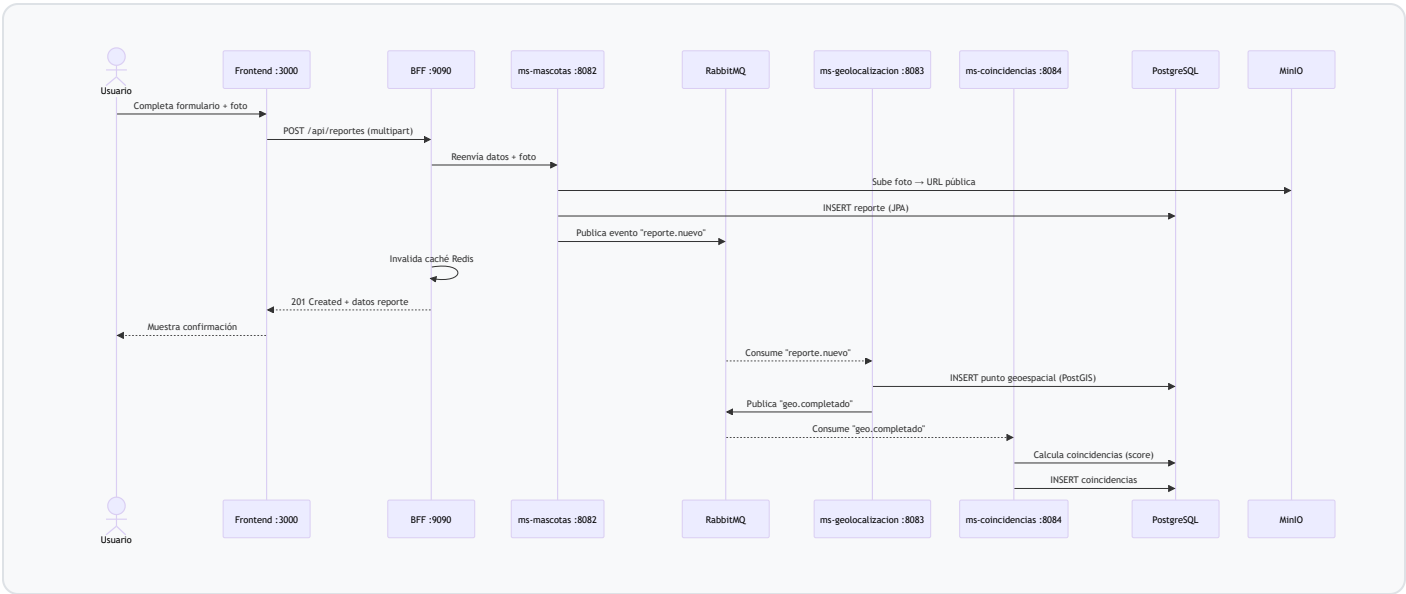
Diagrama de Arquitectura de Microservicios

Sanos y Salvos — DSY1106 Desarrollo Fullstack III — EP3

1. Arquitectura general



2. Flujo de creación de un reporte



3. Componentes y tecnologías

Componente	Tecnología	Puerto	Función
Frontend	React 18 + Vite	:3000	SPA con mapa, formulario de reporte, chat
BFF Service	Spring Boot 3.2	:9090	API Gateway, JWT, Circuit Breaker, caché Redis
auth-service	Spring Boot 3.2	:8081	Registro, login, JWT, rate limiting
ms-mascotas	Spring Boot 3.2	:8082	CRUD reportes, patrón Factory Method
ms-geolocalizacion	Spring Boot 3.2	:8083	PostGIS, heatmap, clusters K-means
ms-coincidencias	Spring Boot 3.2	:8084	Scoring FuzzyWuzzy, matching automático
PostgreSQL + PostGIS	PostgreSQL 15	:5432	Persistencia principal + consultas geoespaciales
Redis	Redis 7	:6379	Caché dashboard y sesiones
RabbitMQ	RabbitMQ 3	:5672	Mensajería asíncrona entre microservicios
MinIO	MinIO	:9000	Almacenamiento fotos (S3-compatible)
Agente n8n	n8n + Claude	:5678	Asistente IA para búsqueda por comuna

4. Patrones de diseño aplicados

Patrón	Dónde	Descripción
Factory Method	ms-mascotas	ReporteFactory crea ReportePerdido o ReporteEncontrado según el tipo
Observer	RabbitMQ	ms-mascotas publica eventos; geo y coincidencias los consumen
Strategy	ms-coincidencias	ScoringService implementa estrategia de scoring multi-criterio
Strategy	ms-geolocalizacion	ClusteringService implementa K-means como estrategia de agrupamiento
Proxy / BFF	bff-service	Backend For Frontend centraliza acceso del frontend a todos los servicios
Circuit Breaker	bff-service	Resilience4j evita cascada de fallos si un microservicio no responde